DMLScope: Uncovering Logical Bugs in Database DML via Approximate Relations

Abstract

...

1 Introduction

2 Background and Motivation

3 Approach

4 Implementation

5 Evaluation

6 Related Work

Logical Bug Detection in DBMSs. A variety of approaches have been proposed to detect logical bugs in DBMSs [4, 6, 11, 13, 14, 29–34]. For example, NoREC [29] transforms an optimizable query into a non-optimizable form and detects semantic inconsistencies by comparing their outputs. TLP [30] divides query predicates into multiple subqueries and verifies that the union of their results is equivalent to the original query. PQS [31] constructs queries expected to retrieve a specific pivot row, while DQE [32] detects logical bugs by comparing whether different SQL queries with the same predicate access the same rows in the database. Pinolo [11] generalizes this paradigm through set-level approximation, where predicates are relaxed or restricted to generate over- and under-approximate queries, and correctness is judged by inclusion or containment relations between result sets. EET [14] applies expression-preserving transformations, and Radar [33] compares query results between databases with and without metadata to identify semantic flaws.

As discussed in Section 1 and Section 2, DMLScope is orthogonal to these prior efforts. Most existing techniques design oracles around query structure and relational operators , and thus excel at exposing bugs in query planning and optimization. However, they are less effective for logical bugs rooted in built-in SQL functions. In contrast, DMLScope targets data-operation-level logical bugs inside built-in SQL function implementations by constructing MRs directly from function properties, enabling function-specific mutations and oracles that remain applicable even when the function is embedded in complex query contexts.

DBMS Test-Case Generation. A variety of approaches have been proposed for generating diverse test cases for DBMSs, with the aim of improving coverage and revealing potential bugs. These techniques [2, 9, 12, 17, 40] typically focus on generating valid and varied SQL queries, but do not specifically target logical bugs. For example, SQLsmith [2] is a grammar-based DBMS fuzzer that embeds SQL grammar rules to generate complex SQL queries. It uses a random walk approach to explore the SQL syntax and generate a wide range of queries. SQUIRREL [40] is a mutation-based DBMS fuzzer which introduces an intermediate representation for SQL queries and models dependencies between SQL statements, enabling the generation of queries that contain multiple SQL operations. Griffin [9] uses a grammar-free mutation approach, where it mutates SQL queries based on DBMS state information encapsulated in a metadata graph. DynSQL [12] takes a dynamic approach by interacting with the DBMS to capture the latest state information, allowing for the incremental generation of valid and complex queries. These techniques aim to prevent semantic errors and improve the diversity of generated queries. In addition to general query generation, some approaches have been specifically designed to aid in bug detection. SQLRight [15] leverages code coverage feedback to enhance test-case generation. QPG [3] takes a different approach by recording the query plans covered during DBMS testing and prioritizing the mutation of queries that trigger new query plans.

These query generation approaches complement DMLScope. They are effective at producing diverse, context-rich queries, while DMLScope can serve as a lightweight add-on that embeds complex built-in function expressions into these queries and applies function property-driven metamorphic relations as test oracles to detect logical bugs in built-in SQL functions.

7 Conclusion

References

[1] Anonymous. 2023. A Detailed Example of Embedding in Rich-context Query (Section 3.3). https://anonymous.4open.science/r/FPMT-DBTesting-D3CA/A%20Detailed%20Example%20of%20Embedding%20in%20Rich-context%20Query%20(Section%203.3).md Accessed: 2026-03-05.

[2] Anse1. 2020.. SQLsmith: A Random SQL Query Generator. https://github.com/anse1/sqlsmith. Accessed Dec 2025.

[3] Jinsheng Ba and Manuel Rigger. 2023. Testing database engines via query plan guidance. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE 23)*. IEEE, 2060–2071.

[4] Jinsheng Ba and Manuel Rigger. 2024. Keep it simple: Testing databases via differential query plans. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–26.

[5] Alibaba Cloud. 2023. PolarDB Product Overview. https://help.aliyun.com/zh/polardb/product-overview/. Accessed Dec 2025.

[6] Wenqian Deng, Jie Liang, Zhiyong Wu, Jingzhou Fu, and Yu Jiang. 2025. Detecting Logic Bugs in DBMSs via Equivalent Data Construction. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–25.

[7] SQLancer developers. 2025. SQLancer: Automated testing to find logic and performance bugs in database systems. https://github.com/sqlancer/sqlancer. Accessed Dec 2025.

[8] MariaDB Foundation. 2023. MariaDB. https://mariadb.org/ Accessed Dec 2025.

[9] Jingzhou Fu, Jie Liang, Zhiyong Wu, Mingzhe Wang, and Yu Jiang. 2022. Griffin: Grammar-free dbms fuzzing. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE 22)*. 1–12.

[10] Jingzhou Fu, Jie Liang, Zhiyong Wu, Yanyang Zhao, Shanshan Li, and Yu Jiang. 2025. Understanding and Detecting SQL Function Bugs: Using Simple Boundary Arguments to Trigger Hundreds of DBMS Bugs. In *Proceedings of the Twentieth European Conference on Computer Systems*. 1061–1076.

[11] Zongyin Hao, Quanfeng Huang, Chengpeng Wang, Jianfeng Wang, Yushan Zhang, Rongxin Wu, and Charles Zhang. 2023. Pinolo: Detecting logical bugs in database management systems with approximate query synthesis. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 345–358.

[12] Zu-Ming Jiang, Jia-Ju Bai, and Zhendong Su. 2023. {DynSQL}: Stateful fuzzing for database management systems with complex and valid {SQL} query generation. In *32nd USENIX Security Symposium (USENIX Security 23)*. 4949–4965.

[13] Zu-Ming Jiang, Si Liu, Manuel Rigger, and Zhendong Su. 2023. Detecting transactional bugs in database engines via {graph-based} oracle construction. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 397–417.

[14] Zu-Ming Jiang and Zhendong Su. 2024. Detecting logic bugs in database engines via equivalent expression transformation. In *18th USENIX Symposium on*

Operating Systems Design and Implementation (OSDI 24). 821–835.
[15] Yu Liang, Song Liu, and Hong Hu. 2022. Detecting logical bugs of {DBMS} with coverage-based guidance. In *31st USENIX Security Symposium (USENIX Security 22)*. 4309–4326.
[16] Li Lin, Zongyin Hao, Chengpeng Wang, Zhuangda Wang, Rongxin Wu, and Gang Fan. 2024. SQLess: Dialect-Agnostic SQL Query Simplification. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 24)*. 743–754.
[17] Li Lin, Qinglin Zhu, Hongqiao Chen, Zhuangda Wang, Rongxin Wu, and Xiaoheng Xie. 2025. QTRAN: Extending Metamorphic-Oracle Based Logical Bug Detection Techniques for Multiple-DBMS Dialect Support. In *Proceedings of the 34nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 25)*. 731–752.
[18] linli1724647576. 2026. ValScope. GitHub repository. https://github.com/linli1724647576/ValScope Accessed: 2026-02-07.
[19] Toby Mao. 2024. SQLGlot: A no-dependency SQL parser, transpiler, optimizer, and engine. https://github.com/tobymao/sqlglot Accessed Dec 2025.
[20] MySQL. 2023. MySQL. https://www.mysql.com/ Accessed Dec 2025.
[21] OceanBase. 2023. OceanBase Open Source. https://www.oceanbase.com/product/opensource Accessed Dec 2025.
[22] Oracle. 2026. Chapter 6: Adding Functions to MySQL. https://dev.mysql.com/doc/extending-mysql/8.3/en/adding-functions.html. Accessed: 2026-01-23.
[23] Oracle. 2026. Functions and Operators. https://dev.mysql.com/doc/refman/8.0/en/functions.html. Accessed: 2026-01-23.
[24] Percona. 2023. Percona - Open Source Database Software and Services. https://www.percona.com/ Accessed Dec 2025.
[25] Percona. 2026. PS-10441: Queries exposing a logical bug in built-in SQL function. https://perconadev.atlassian.net/browse/PS-10441. Accessed: 2026-02-04.
[26] PingCap. 2022. go-randgen. https://github.com/pingcap/go-randgen Accessed Dec 2025.
[27] PingCAP. 2023. TiDB. https://www.pingcap.com/tidb/ Accessed Dec 2025.
[28] PolarDB. 2023. Issue 259: Bug Report for PolarDBX-SQL. https://github.com/polardb/polardbx-sql/issues/259 Accessed: 2023-12-07.
[29] Manuel Rigger and Zhendong Su. 2020. Detecting optimization bugs in database engines via non-optimizing reference engine construction. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1140–1152.
[30] Manuel Rigger and Zhendong Su. 2020. Finding bugs in database systems via query partitioning. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–30.
[31] Manuel Rigger and Zhendong Su. 2020. Testing database engines via pivoted query synthesis. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 667–682.
[32] Jiansen Song, Wensheng Dou, Ziyu Cui, Qianwang Dai, Wei Wang, Jun Wei, Hua Zhong, and Tao Huang. 2023. Testing database systems via differential query execution. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE 23)*. IEEE, 2072–2084.
[33] Jiansen Song, Wensheng Dou, Yu Gao, Ziyu Cui, Yingying Zheng, Dong Wang, Wei Wang, Jun Wei, and Tao Huang. 2024. Detecting Metadata-Related Logic Bugs in Database Systems via Raw Database Construction. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1884–1897.
[34] Xiu Tang, Sai Wu, Dongxiang Zhang, Feifei Li, and Gang Chen. 2023. Detecting logic bugs of join optimizations in dbms. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–26.
[35] The PostgreSQL Global Development Group. 2015. Security Update Release 16.15. https://www.postgresql.org/about/news/2015-10-08-security-update-release-1615/. Accessed: 2026-02-03.
[36] The PostgreSQL Global Development Group. 2026. Chapter 9. Functions and Operators. https://www.postgresql.org/docs/current/functions.html. Accessed: 2026-01-23.
[37] The PostgreSQL Global Development Group. 2026. User-Defined Functions. https://www.postgresql.org/docs/current/xfunc.html. Accessed: 2026-01-23.
[38] Rick F. Van Der Lans. 1989. *The SQL Standard: A Complete Guide Reference*. Prentice Hall International (UK) Ltd.
[39] Wikipedia. 2026. Databases. https://en.wikipedia.org/wiki/Database. Accessed: 2026-01-23.
[40] Rui Zhong, Yongheng Chen, Hong Hu, Hangfan Zhang, Wenke Lee, and Dinghao Wu. 2020. Squirrel: Testing database management systems with language validity and coverage feedback. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 955–970.